

The Complete Angular & TypeScript Masterclass

Welcome to the comprehensive walkthrough for modern web development using Angular. This guide takes you from the foundational concepts of ES6 and TypeScript through the advanced architecture of an Angular application, concluding with real-world patterns like RxJS data streams and JWT Authentication.

1. ES6 Basics

Before diving into Angular, it is crucial to understand the modern JavaScript (ES6+) features that power it.

- **let/const:** `var` is function-scoped and prone to hoisting bugs. `let` allows you to declare block-scoped variables that can change. `const` declares block-scoped variables that cannot be reassigned (though objects/arrays assigned to `const` can still be mutated).
- **Template Literals:** Enclosed by backticks (```), they allow multi-line strings and embedded expressions using `${expression}`.
- **Destructuring:** A syntax that allows unpacking values from arrays or properties from objects into distinct variables.

```
const { name, age } = userObj;  
const [first, second] = myArray;
```

- **Spread Operator (`...`):** Expands an iterable into more elements. Useful for cloning objects or merging arrays.

```
const newArray = [...oldArray, 4, 5];
```

2. TypeScript Foundations

TypeScript is a superset of JavaScript that adds static typing, enhancing tooling and error checking.

Basics

- **Types & Arrays:** Basic types include `string`, `number`, `boolean`, and `any`. Arrays are typed like `string[]` or `Array<number>`.
- **Functions:** You can explicitly define parameter types and return types.

```
function add(a: number, b: number): number { return a + b; }
```
- **Interfaces:** They define the shape or contract of an object.

```
interface User {  
  id: number;  
  name: string;  
}
```

Advanced TypeScript

- **Unions (|)**: Allows a value to be one of multiple types (e.g., `string | number`).
- **Optionals (?)**: Denotes that a property or parameter is not strictly required.
- **Narrowing**: Using type guards (like `typeof` or `instanceof`) to safely refine a union type into a specific type before operating on it.
- **TSConfig Basics**: The `tsconfig.json` file configures the TypeScript compiler, specifying the target JS version, strictness levels, and module resolution rules.

3. Introducing Angular

Angular is a platform and framework for building single-page client applications using HTML and TypeScript.

- **Central Features**: Component-based architecture, powerful CLI, two-way data binding, dependency injection, and comprehensive routing.
- **Architecture & Anatomy**: An Angular app is a tree of components. The `angular.json` file manages workspace configuration. The `src/main.ts` is the bootstrap file that launches the root module (`AppModule`).
- **CLI Commands**:
 - `ng new app-name` : Creates a project.
 - `ng serve` : Runs the app locally.
 - `ng build` : Compiles the app into an output directory for deployment.

4. Components (Basics & Advanced)

Components are the fundamental building blocks of Angular applications.

Creating & Registering

Generate a component using `ng generate component name`. The `@Component` decorator provides metadata (selector, templateUrl, styleUrls). The component must be declared in an `@NgModule` to be usable.

Lifecycle Hooks

Angular manages the creation, rendering, and destruction of components. Key hooks include:

- `ngOnInit()` : Called once after initialization. Ideal for data fetching.
- `ngOnChanges()` : Called when input properties change.
- `ngOnDestroy()` : Called before the component is destroyed. Critical for cleanup (unsubscribing from observables).

5. Templates, Data Binding & Directives

Angular templates connect the component class to the DOM.

- **Interpolation:** Mustache syntax `{{ value }}` binds class properties to the template.
- **Property & Event Binding:** `[property]="value"` sets DOM properties. `(event)="handler()"` listens to user actions.
- **Directives:**
 - *Structural:* Alter layout (`*ngIf` , `*ngFor` , `*ngSwitch`).
 - *Attribute:* Alter appearance/behavior (`[ngStyle]` , `[ngClass]`).

Parent-Child Communication: Use `@Input()` to pass data down to a child component, and `@Output()` with `EventEmitter` to send events up to the parent.

6. Handling Forms

Angular provides two distinct approaches to handling user input.

Template-Driven Forms

Best for simple forms. Relies heavily on HTML directives (`[(ngModel)]` for two-way binding). Requires `FormsModule` . Validation state is tracked via CSS classes: `ng-pristine` , `ng-dirty` , and `ng-touched` .

Reactive Forms

Best for complex, dynamic forms. Logic is built in the TypeScript class using `FormGroup` and `FormControl` . Requires `ReactiveFormsModule` .

```
this.myForm = new FormGroup({
  email: new FormControl('', [Validators.required, Validators.email])
});
```

7. Services, Dependency Injection & Pipes

- **Services:** Classes decorated with `@Injectable()` used to share logic or data across components.
- **Dependency Injection (DI):** Angular instantiates and injects these services into component constructors automatically.
- **Pipes:** Used to transform data in templates (e.g., `{{ dateObj | date:'shortDate' }}`). Pipes can be chained. Custom filter pipes can be created, though Angular recommends filtering arrays in the component class for performance.

8. Routing & Modules

Routing enables navigation from one view to the next as users perform application tasks.

- **Router Setup:** Defined in an array of routes mapping paths to components. Displayed via `<router-outlet>`.
- **Parameters:**
 - *Route Params:* Used for mandatory data (e.g., `/users/:id`). Accessed via `ActivatedRoute.snapshot.paramMap`.
 - *Query Params:* Used for optional data (e.g., `/search?q=angular`).
- **NgModules:** Group related code. Feature modules keep the app organized and enable Lazy Loading (loading chunks of code only when a user navigates to that route).

9. HTTP Client & RxJS

Angular applications communicate with backend services using the `HttpClient` service, which relies heavily on RxJS.

- **Making Requests:** `this.http.get<Type>(url)` returns an *Observable*.
- **RxJS Observables:** Unlike Promises (which fire once), Observables emit streams of data over time and can be cancelled.

- **Operators:** You can transform data streams using operators like `map` (transform data), `filter` (ignore specific data), and `catchError` (handle HTTP failures) inside the `.pipe()` method before calling `.subscribe()`.

10. Angular Authentication (JWT)

Securing an Angular app usually involves JSON Web Tokens (JWT).

- **Login Flow:** The user submits credentials. The server verifies and returns a JWT. The Angular app stores this token (e.g., in `localStorage`).
- **HTTP Interceptors:** A service that catches every outgoing HTTP request to automatically clone it and attach the JWT in the `Authorization` header.
- **Route Guards:** `CanActivate` guards prevent unauthorized users from navigating to protected routes.
- **Logout:** Clears the token from storage and redirects the user back to the login screen.